

# ndgrid

Rectangular grid in N-D space

## Syntax

```
[X1,X2,...,Xn] = ndgrid(x1,x2,...,xn)
[X1,X2,...,Xn] = ndgrid(xg)
```

## Description

[X1,X2,...,Xn] = ndgrid(x1,x2,...,xn) replicates the grid vectors x1,x2,...,xn to produce an n-dimensional full grid. [example](#)

[X1,X2,...,Xn] = ndgrid(xg) specifies a single grid vector xg to use for all dimensions. The number of output arguments you specify determines the dimensionality n of the output.

## Examples

[collapse all](#)

### Create 2-D Grid

Create a 2-D grid from the vectors [1 3 5 7 9 11 13 15 17 19] and [2 4 6 8 10 12].

Open in MATLAB Online

Copy Command

```
[X,Y] = ndgrid(1:2:19,2:2:12)
```

Get ▼

X = 10x6

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 1  | 1  | 1  | 1  | 1  | 1  |
| 3  | 3  | 3  | 3  | 3  | 3  |
| 5  | 5  | 5  | 5  | 5  | 5  |
| 7  | 7  | 7  | 7  | 7  | 7  |
| 9  | 9  | 9  | 9  | 9  | 9  |
| 11 | 11 | 11 | 11 | 11 | 11 |
| 13 | 13 | 13 | 13 | 13 | 13 |
| 15 | 15 | 15 | 15 | 15 | 15 |
| 17 | 17 | 17 | 17 | 17 | 17 |
| 19 | 19 | 19 | 19 | 19 | 19 |

Y = 10x6

|   |   |   |   |    |    |
|---|---|---|---|----|----|
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |

|   |   |   |   |    |    |
|---|---|---|---|----|----|
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |
| 2 | 4 | 6 | 8 | 10 | 12 |

### Refine Grid Using Interpolation

Create a rectangular grid and calculate function values on the grid. Interpolate between the assigned values to refine the grid.

Create a coarse grid for  $(x, y)$ , where the range of  $x$  is  $[-6, 6]$  and the range of  $y$  is  $[-3, 3]$ .

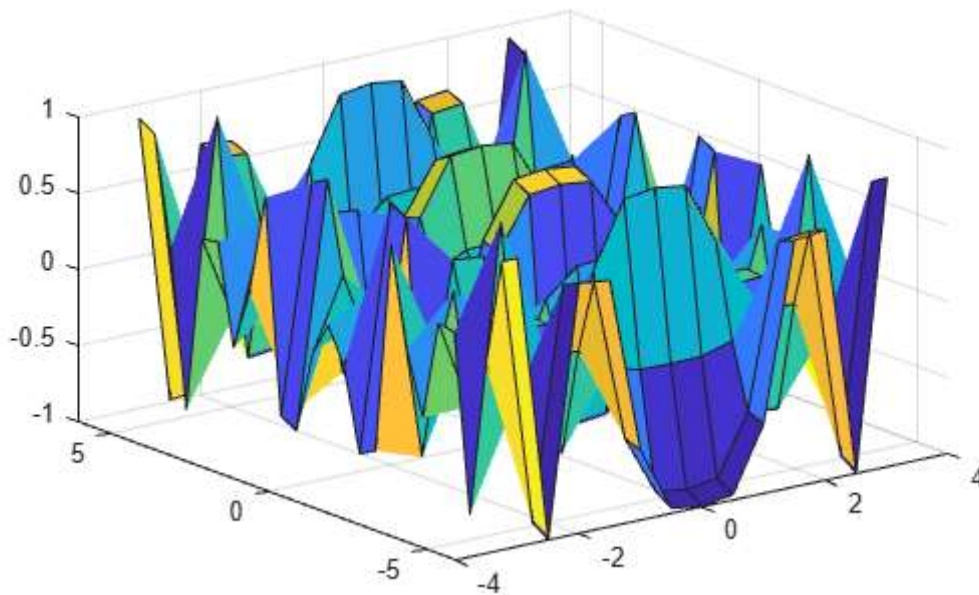
[Open in MATLAB Online](#)
[Copy Command](#)

```
[X,Y] = ndgrid(-6:0.5:6,-3:0.5:3);
```

[Get](#)

Evaluate the function at the locations defined in the grid. Then, visualize the function using a surface plot. Alternatively, since R2016b, you can use implicit expansion for this task.

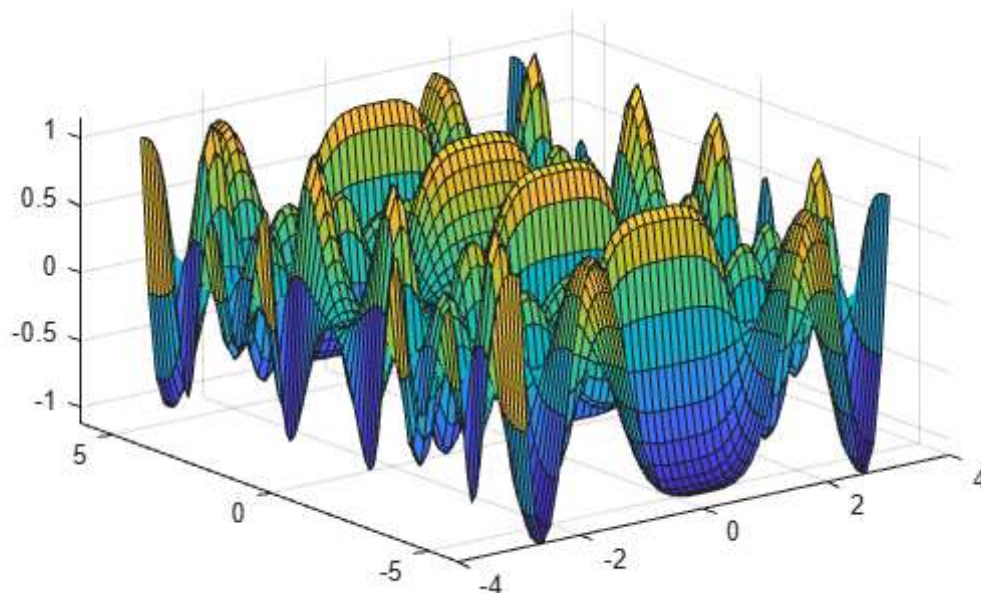
```
f = sin(X.^2) .* cos(Y.^2);  
surf(Y,X,f)
```

[Get](#)


Interpolate between the points on a more refined grid  $(X_q, Y_q)$ . Then, visualize the interpolated values using a surface plot.

```
[Xq,Yq] = ndgrid(-6:0.125:6,-3:0.125:3);  
F = interpn(X,Y,f,Xq,Yq,"spline");  
surf(Yq,Xq,F)
```

[Get](#)

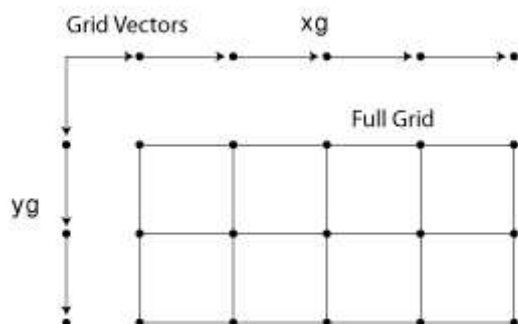


## Input Arguments

[collapse all](#)

**$x_1, x_2, \dots, x_n$  — Grid vectors (as separate arguments)**  
vectors

Grid vectors, specified as vectors containing grid coordinates for each dimension. The grid vectors implicitly define the grid. For example, in 2-D:



**Data Types:** single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

**Complex Number Support:** Yes

**$xg$  — Grid vector for all dimensions**  
vectors

Grid vector for all dimensions, specified as a vector containing grid coordinates. `ndgrid` uses `xg` as the grid vector for each dimension.

**Data Types:** single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64**Complex Number Support:** Yes

## Output Arguments

[collapse all](#)**X1,X2,...,Xn — Full grid representation**

array

Full grid representation, returned as separate arrays. For each output array  $X_i$ , the  $i$ th dimension contains copies of the grid vector  $x_i$ .

## More About

[collapse all](#)

### Convert Between meshgrid and ndgrid Formats

meshgrid and ndgrid create grids using different output formats. Specifically, the first two dimensions of a grid created using one of these functions are swapped when compared to the other grid format. Some MATLAB<sup>®</sup> functions use grids in meshgrid format, while others use ndgrid format, so it is common to convert grids between the two formats.

You can convert between these grid formats using [pagetranspose](#) (as of R2020b) or [permute](#) to swap the first two dimensions of the grid arrays. For example, create a 3-D grid with meshgrid.

```
[X,Y,Z] = meshgrid(1:4,1:3,1:2);
```

Now transpose the first two dimensions of each grid array to convert the grid to ndgrid format, and compare the results against the outputs from ndgrid.

```
Xt = pagetranspose(X);
Yt = pagetranspose(Y);
Zt = pagetranspose(Z);
[Xn,Yn,Zn] = ndgrid(1:4,1:3,1:2);
isequal(Xt,Xn) & isequal(Yt,Yn) & isequal(Zt,Zn)
```

```
ans =
```

```
logical
```

```
1
```

Using pagetranspose is equivalent to permuting the first two dimensions while leaving other dimensions the same. You can also perform this operation using permute( $X$ , [2 1 3:ndims( $X$ )]).

## Extended Capabilities

[expand all](#)

### C/C++ Code Generation

Generate C and C++ code using MATLAB<sup>®</sup> Coder<sup>™</sup>.

### GPU Code Generation

Generate CUDA<sup>®</sup> code for NVIDIA<sup>®</sup> GPUs using GPU Coder<sup>™</sup>.

**Thread-Based Environment**

Run code in the background using MATLAB® backgroundPool or accelerate code with Parallel Computing Toolbox™ ThreadPool.

**GPU Arrays**

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

**Distributed Arrays**

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

## Version History

---

Introduced before R2006a

## See Also

---

[griddedInterpolant](#) | [mesh](#) | [meshgrid](#) | [combinations](#) | [surf](#)

## Topics

[Interpolating Gridded Data](#)

YOU MIGHT ALSO BE INTERESTED IN

[Integrating MATLAB with Python](#)

Use MATLAB® as a computational engine in Python®, or use Python libraries and models in MATLAB.

[Learn more](#)